

Toward Generalizable Robotics Pushing

Steve Xing¹

I. INTRODUCTION

HUMANS demonstrate an ability to generalize pushing varieties of objects. Oftentimes, humans do not need to take into account the specific parameters of the object and environment such as friction coefficients. Towards making robots more generalizable when manipulating objects in the environment like humans, there should be a universal method to allow robots to push a variety of objects with minimal modeling and parameter considerations.

II. PROBLEM DEFINITION

The problem definition is as follows: a rectangular object of which the friction is not known, only dimensions, is placed in the workspace in an initial position near the robot. This rectangular object, by using the robot's end-effector rod to push the object by tiny increments in a chosen direction, should reach a final goal state at some orientation and rotation. The final outcome is to have minimal modeling of the object and world dynamics, and still achieve successful pushing towards a goal.

III. APPROACH OUTLINE

A. Backend Control

The backend control consists of hardware-software interface, inverse-kinematics planning, and trajectory planning. The hardware-software interface will be used to communicate high-level commands sent in code into low-level control signals to each joint's motor controller. Inverse-kinematics planning will be used to command the end-effector rod to desired coordinates. Trajectory planning will be used to smoothly interpolate between a current and future point as to achieve proper and consistent pushing.

B. State estimation

In order to assess how well an algorithm does at pushing an object, the object's state should be known at any point of time. The method used to estimate the object's state should be consistent and relatively accurate, so that any scoring-metric used in control will also be consistent. Information about the object's position and rotation should be known, and be used as inputs to the scoring-metric function.

C. Push-sequence execution

This will be the main running loop that will use the robot's end-effector to push the object for it to reach a goal position. This loop will be responsible for estimating the state of the object, pick the best direction to push in, and move the end-effector in the push direction. The loop breaks once the object reaches the goal position.

IV. METHODS

A. Overview of Proposed Method

My proposed method is as follows: The robot moves the end-effector from its current position to a known zero position. The top-down view camera opens for a single frame, capturing two ARUCO markers, one placed on the moving object, and one placed on a static board. The ARUCO markers' poses are captured, and a scoring function to capture distance and orientation from a goal position uses the returned x, y, and z-axis rotation values. With object's position known, MPC algorithm is run over a horizon that predicts future positions and returns next best direction to push in. Then, Inverse kinematics and trajectory planning is used to go from current end-effector position to target position based on direction to push in. The current position of end-effector is updated to new position after push, and the loop repeats.

B. Technical details of the approach and algorithms

1) *Backend Control*: The motors controllers are the **MjBot S1s**. Each motor controller controls a single motor, and each is assigned an ID on a CAN-Bus on the Raspberry-Pi. The IDs start from 1 and end at 5 inclusive, denoting the robot's 5-degree-of-freedom setup. I used a library, **pi3hat_hardware_interface**[1], that abstracts sending high level joint commands to each motor controller CAN ID, and handles the necessary feedforward and PID signals. In order to interface with the low-level motor controllers, I choose to use `ros2_control`. By doing so, I can also define how each actuator joint is related to the motor. For example, my shoulder joint is defined in my `ros2_control XML` as a 9:1 speed-reduction with limits located at 2.42 radians and -1.7 radians, and accepts position, velocity, and effort commands. While the majority of the process was streamlined, I had greater difficulty modeling the end-effector, which is a semi-differential drive setup driven by two motors. For the semi-differential drive setup, I used `ros2_control`'s Differential Transmission interface. Here, instead of each motor being assigned to an individual joint actuator, I needed to define the pair of motors as actuators that, when spinning in the same direction caused a change in end-effector pitch, while spinning in opposite directions caused a change in end-effector roll.

As the robot is a custom design, there is no existing written analytical inverse kinematics solution for it that works fast, only numerical algorithms. Regardless, the various IK implementations I explored were the following:

- (a) *ROS2 Moveit2*: This library includes motion planning and inverse kinematics implementations, albeit a heavy library.

As I was running my entire codebase on a single Raspberry Pi without enough compute resources, this choice proved to be unrealistic for the time being, as installation and live usage continually crashed the system.

- (b) **Orocos KDL IK**[2]: I tested Orocos' KDL IK library next, using this repository written by Arne Baeyens as blueprint. This library proved to be a minimal setup to get quick IK solutions for various end-effector positions on my robot. It uses the `/robot_description` topic to get the robot's structure and form a resulting IK solution based on the Orocos KDL IK implementation. Here, I am using KDL's Levenberg-Marquardt IK-solver implementation. I ultimately chose to build my system on this library as a result.
- (c) **Analytical transformation matrices**: I wrote out the transformation matrices between each joint. With these matrices determined, the analytical solution for an end-effector position should be obtainable. Since in my setup, the end-effector's z-position is fixed, I am able to match my 5-DOF robot setup with a 5-DOF end-effector orientation, making the system redundant. This could be a good alternative to the Orocos KDL IK method, but I did not have adequate time to solve for the explicit joint solutions.

I also implemented trajectory-planning, which will help move the end-effector smoothly between its current position and a target position with minimal jerkiness. This allows for consistent pushing patterns and minimize any sudden accelerations of the object that would cause it to lose contact with the end-effector. Specifically, I linearly interpolate the x and y positions of the end-effector for 400 steps over 2 seconds—the z-axis is held constant and does not need interpolation. At each step, an IK is run on the target (x,y,constant z) position, and the resulting 5 joint angles are published to the robot's motor controllers.

2) **State estimation**: The overarching goal for state estimation seeks to get a consistent method of getting the object's position by using only a single over-head Raspberry Pi Camera. The camera provides a 640x480 frame picture, and runs at 30 frames per second. The camera does not have any depth-estimation capabilities. I explored two main methods of estimating state:

- (a) **Color-based Shape Contour**: This was the initial minimal setup for state estimation. In this setup, I applied yellow tape that entirely covered the top surface of the object. In the code, I first threshold the image to capture yellow colors between RGB values of (20, 100, 100) and (40, 255, 255). I then used a 7x7 kernel to smooth out any noise or gaps inside of the yellow contour, resulting in a consistent mask shape. I applied OpenCV's `findContours` method, putting the contour output into `minAreaRect` to obtain an estimate of the object's center xyz position. While the object was consistently detected every frame and could be used to get an estimate of the object's pose, it was inconsistent. Namely, based on different lighting conditions and shadows, parts of the yellow contour would "shift" and result in shifted xyz position estimates despite the object having moved 10mm from its previous position.

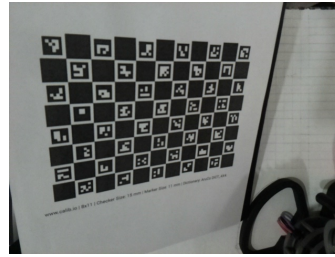


Fig. 1. Charuco Calibration Board

- (b) **ARUCO Marker**: In this setup, two ARUCO marker from the DICT 4X4_50 set of markers are taken, one for the object, and one for the static base to represent a fixed environment space. One ARUCO marker with ID 0 is placed and taped on top of the object, centered with the center of the object. The other ARUCO marker with ID 1 is placed some distance away from the starting position of the end-effector. From a top-down view, the entirety of both ARUCO markers are visible. I use OpenCV's `detectMarkers` method, obtain each marker's ID, and enumerate through the IDs in order to get the xyz and axis-angles. As the camera's intrinsics and distortion coefficients are relevant in this process, I needed to also calibrate my camera. To do this, I printed out a Charuco board on A4 paper with the parameters: 200mm x 150mm board, with 8x11 number of squares, where each black square was 15mm, each ARUCO marker was 11mm, and using ARUCO markers from the 4X4_DICT collection. I took 60 different pictures of the board from the overhead Pi-camera, with the board oriented at different angles. I then utilized Matěj Šmíd's `video2calibration` repository[3] to get the final intrinsic and distortion parameters. This method is reliable under most lighting conditions, and the center of the ARUCO marker is consistently detected and is relatively immune to shifts in perspective.

3) **Push-sequence execution**: This is the main loop that allows the end-effector to push the object, for the camera to be opened, and for the state of the object to be estimated before a new decision for push direction is made. I use two primary control methods to decide what next direction to push in: MPC and a Heuristic.

- (a) **MPC**: In this method, I use a planning horizon of 4 steps. At each step, the action space has size 3: push forward by 10mm, push diagonal to the left by 10mm, and push diagonally to the right by 10mm, relative to the robot's base orientation. I predict the next state given this push direction choice using `predictNextState`. For push forward by 10mm, or encoded to action 0, the end-effector moves forward by 10mm relative to the robot base, which results in the object's coordinates changing by 7.07mm in the X and Y directions, relative to the fixed ARUCO marker frame. For a left diagonal push by 10mm, or encoded to action 1, the end-effector moves in the X any Y directions by 7.07mm, which results the object's coordinates changing by 10mm in the Y direction, relative to the fixed ARUCO marker frame. For a right diagonal

push by 10mm, or encoded to action 7, the end-effector moves in the X any negative Y directions by 7.07mm, which results the object's coordinates changing by 10mm in the X direction, relative to the fixed ARUCO marker frame. After the next state is obtained, I use a scoring function, `gradeDistanceMetric`, which takes into account the object's rotation and xy position relative to a goal position. I employ a greedy method to choose the minimal resulting cost from choosing one of the 0, 1, 7 action. This next state is fed into the next MPC horizon step.

- (b) **Heuristic:** In this method, I only need to choose an action based on the current state. Namely, the logic is as follows: if the rotation of the object is within +5 or -5 degrees from a desired heading—which is 45 degrees relative to the fixed ARUCO marker frame—then use the `predictNextState` function to choose the action that will result in the minimal next object distance. If the rotation of the object is greater than +5 degrees from the desired heading, I will choose action 7, assuming that a push in the +X direction in fixed ARUCO marker frame will fix this error. On the other hand, if the rotation of the object is less than -5 degrees from the heading, I will choose action 0, assuming a push in the +Y direction in fixed ARUCO marker frame will correct the error.

V. EXPERIMENTS

A. Settings and Implementation details

The experiment setup is pictured in Figure 2. Here, there is a white fixed base on which the robot base, object, and ARUCO marker with ID 1 (for a fixed reference) sit. The robot base sits 200mm in the -X and -Y directions from the ARUCO marker taped to the board, where +X direction is to the left-hand direction in Figure 2, with +Z pointing upwards. The robot's base is then oriented 45 degrees such that +X direction in the robot's base frame points towards the ARUCO marker. The object is a 3d-printed rectangle, with dimensions of 10x35x50mm. The ARUCO marker with ID 0 is placed on the side of the object's surface with dimension 10 x 35mm. The robot is first commanded to move to a location where the end-effector rod is pointing down, into the board, as pictured in Fig 2. The object is initially placed so that the middle edge of the 50mm dimension is in contact with the end-effector. The execution loop then begins running from this step. Since the system still remains relatively inaccurate, I modified the scoring function such that as long as the object is some distance away from the goal position, and in a proper orientation of 45 degrees with a 3 degree margin relative to the fixed goal frame, it will satisfy as a success. Otherwise, after 25 iterations, or pushes, the loop breaks and counts as a failure.

B. Details of the experiments and results

The experiment was broken into two parts, one testing the results of MPC based pushing, and one testing the results of Heuristics based pushing. For evaluation, the experiment was run 10 independent times for each method, and success rate calculated. The success distance is currently set at 35mm away from the goal position.

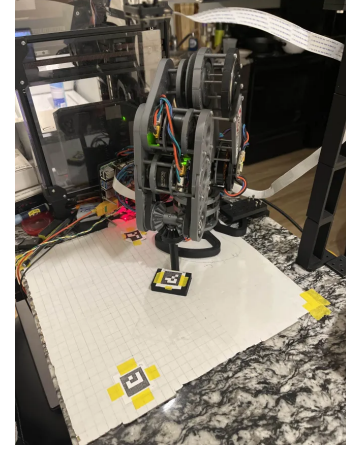


Fig. 2. Setup environment of robot, camera, static board environment, and object

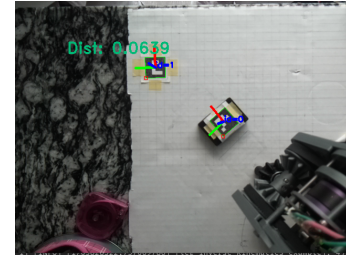


Fig. 3. Example of the scene midway through the push process. The state pictured is 10 iterations into the process

1) **MPC - Results:** The Heuristics-based results are shown in Figure 5. The average distance of the final object position is 0.0530529m or 53mm, with an end heading of 15 degrees. The figure shows that the trajectory is pushing on both x and y axes diagonally, but still results in failures by running out of the 25 iteration steps. In fact, from watching the

2) **Heuristics - Results:** The Heuristics-based results are shown in Figure 5. The average distance of the final object position is 0.0267005m or 26mm, with an end heading of 39 degrees. The figure shows that the trajectories are correctly tending towards the goal position with x and y usually moving diagonally instead of along just a single axis.

C. Analysis of the results

The MPC-based trajectory results are visualized in Figure 6. Overall, MPC fails to make any meaningful push near the



Fig. 4. Example of the scene at the end of the push process. The state pictured is 25 iterations into the process

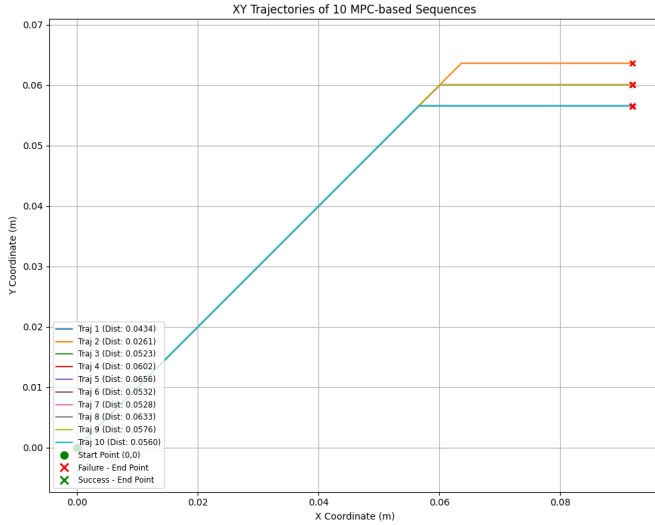


Fig. 5. Plot for end-effector position from MPC-based pushing. The accuracy is 0.0 of 10 evaluation runs. Failed trajectories are denoted with an ending red X-mark, of which there are 10.

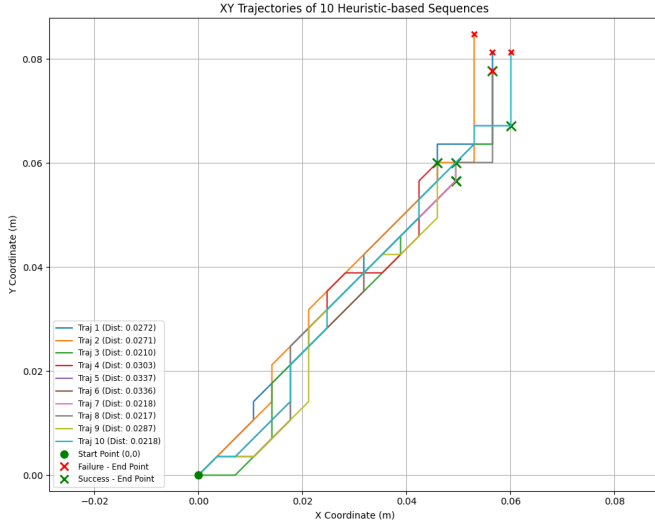


Fig. 6. Plot for end-effector position from Heuristic-based pushing. The accuracy is 0.7 of 10 evaluation runs. Failed trajectories are denoted with an ending red X-mark, of which there are 3.

end of the trajectory, and entirely loses contact with the object around 18 pushes in. This can be explained by the fact that object begins biasing towards 0 degrees away from 45 degrees heading at around 10 pushes in. The MPC algorithm, likely due to a badly-tuned or incorrect-assumption-based next-state prediction, fails to act early. The error accumulation thus continually grows and results in MPC algorithm making the wrong decision too late. As the average final object distance of 53mm is relatively high and object heading ends up at 15 degrees, it is clear that the current MPC implementation fails to make meaningful progress. One potential reason for the overall failure is that the next-state prediction is not robust enough. Namely, in implementing it, I took in assumptions that the end-effector would always be in contact with the object, and the object would not deviate too far from the

desired 45 degree heading. Therefore, I would only need to worry about the x and y positions changing consistently based on the direction the end-effector pushes in. In essence, I heavily simplified the assumptions of the dynamics of the system. Although this is towards the final goal of the project, I potentially might consider other solutions such as learning models, as discussed in the Conclusion. I unfortunately did not have sufficient time to successfully create and implement an algorithm that took into account both the rotation of the object along with predicted rotation dynamics from a push in a certain direction. I believe I would be able to analyze the Heuristics-based trajectory behavior and form an understanding of how to best model this next-state prediction that attempted to estimate rotation change, not just position change.

The Heuristics-based trajectory results are visualized in Figure 5. Overall, Heuristics seems to result in much higher accuracy compared to MPC. One of the main failure modes is that the end-effector is unable to correct the angle of the object towards 45 degrees near the end of the trajectory, running out of steps. This may be due to the reason that the arm begins to sag under greater weight as it is extended out, and where pushing becomes somewhat inconsistent. At some points the end-effector could also lose contact with the object. The average end distance is 0.0267005m, whilst the average end rotation is 39 degrees, showing that the end-effector can comfortably push the object withing range of the goal of 0.035m, but tends to push the object that biases the rotation towards 0 degrees, or along the x-axis with respect to the fixed ARUCO frame. The most common action aside from 0 is 7, or a push to bias the object's rotation more towards the y-axis with respect to the fixed ARUCO frame. This shows that throughout the entire push process, the robot needs to continually correct the object's heading away from the x-axis in fixed frame. Some modifications might be made so that the end-effector can push even greater distances through the 1 or 7 actions in order to induce greater rotation correction. One advantage that Heuristics-based algorithm benefits over MPC might be the immediate response to small errors in the object's rotation. By actively using 1 or 7 rather than settling with a default straight forward 0-action push, these tiny errors are addressed early on and can prevent higher error accumulation.

One pattern common across both MPC and Heuristics evaluations is that the object tends towards the +X axis of the board rather than +Y or both +XY. This might be due to a mechanical setup error, especially if the mounting of the robot's base is not precisely 45 degrees with respect to the board. Both MPC and Heuristics tend to choose the action of 0, or pushing forward. MPC tends to choose this more frequently, but still failing, showing that the simple next-state prediction dynamics may be incorrectly or insufficiently modeled.

VI. CONCLUSION

A. Discussion

Overall, the Heuristics-based method seems to be more reliable than the MPC-based method. However, this is not to discount potential mechanical or software setup errors that might compound onto a high-failure algorithm. As with

working on a custom system such as my robot, much of the implementation for algorithms could only work after obtaining a working hardware-software interface and task-environment setup. Heuristics may have found higher success due to the fact that it actively corrects for the rotation error very early on, whilst the MPC next-state prediction fails to take into account how the object's rotation can impact assumptions about how the X and Y coordinates should move.

B. Future Work

In the future, it may be better to properly plan out required hardware and software setup, fully complete those goals to be sure that their contributing-errors are minimized during experiments. It may also be better to develop these algorithms that rely on consistent robot dynamics on a much more mature system, such as UR5e robots. Furthermore, it may still be helpful to simulate many experiments before deployment onto a real world system, as it minimizes things such as certain joint commands being sent to the robot controllers resulting in undesirable state. It may also uncover whether libraries I use, such as Orocos' KDL Inverse Kinematics, can fully account for any end-effector orientation on a custom-robot system. As for the MPC method, which did not seem to provide meaningful success, it might be better to hand off next-state dynamics prediction to things such as a learning model, which takes in robot delta joint states and state-estimation metrics of the environment. Modeling a very simple dynamic pattern by hand seems unreliable and can be highly inaccurate.

Given greater time to iterate on the solution for the project on my custom robot platform, I will likely first get certain guarantees that my Inverse Kinematics, actuator PIDs, and trajectory-planners are correct and consistent. Afterwards, I will work to ensure that my environment setup, including camera mounting solutions, positioning of robot base, camera, and environment board base are correct and remain static throughout different runs. This includes running unit tests for expected behavior. Furthermore, I will consider getting an accurate model of the robot in simulation itself, as this will minimize any potential differences with real-world planning, and serve as a way to validate that my IK inputs are reachable. For the algorithm implementations, I will likely take greater time to consider whether using learning-models could serve best for next-state dynamics predictions, and use the relatively successful Heuristics-based trajectories as training data.

C. Codebase

Found at: https://github.com/Zelo-S/cs498_robotarm

REFERENCES

- [1] Gabrael Levine, `pi3hat_hardware_interface`, https://github.com/G-Levine/pi3hat_hardware_interface/tree/main
- [2] Arne Baeyens, `kdl-tutorials`, <https://github.com/abaeyens/kdl-tutorials/tree/main>
- [3] Matěj Šmíd, `video2calibration`, <https://github.com/smidm/video2calibration>